

discussion of their components over the following months in a series of subsequent articles. Understanding the changing landscape of heterogeneous systems would benefit those interested in next-generation disruptive computer architecture changes, as this brings in rich opportunities in the open source hardware and software ecosystem. Towards that, let us start by looking at how the computer architecture landscape has evolved over the years,

The evolution of computer architecture started out with computers that had single cores, before reaching today's level of heterogeneous multi-core scale-out systems. As shown in Figure 1, from 1975 to 2005, the industry accomplished something phenomenal—getting PCs on desks, in millions of homes, and in virtually every pocket. The year 2005 saw a shift to computers with multi-core architecture, where parallel computing became a possibility to cater to the ongoing demand from users for better performance. It was in 2011 that we saw these parallel supercomputers with multi-cores coming out in popular form factors like tablets and smartphones.

The multi-core era

Multi-core systems can provide high energy efficiency since they allow the clock frequency and supply voltage to be reduced together to dramatically reduce power dissipation during periods when the full rate of computation is not needed.

A multi-core CPU combines multiple independent execution units into one processor chip, in order to execute multiple instructions in a truly parallel fashion. The cores of a multi-core processor are sometimes also denoted as processing elements or computational engines. According to Flynn's taxonomy, the resulting systems are true multiple-instruction multiple-data (MIMD) machines, able to process multiple threads of execution at the same time.

The multi-core era saw two classes of systems — homogeneous multi-core systems (SMPs) and heterogeneous multi-core systems (HMPs).

Homogeneous multi-core systems

A symmetric multi-processor system (SMP) has a centralised shared memory called the main memory (MM) operating under a single operating system with two or more processors of the same kind. It is a tightly coupled multi-processor system with a pool of homogeneous processors

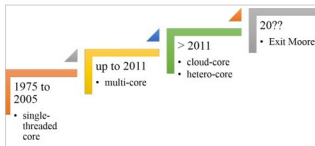


Figure 1: Timeline showing the evolution of computer architecture

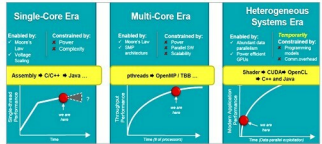


Figure 2: Depiction of the various computing eras (Source: <http://www.hsafoundation.com/>)

running independently. Each processor executes different programs and works on different data, with the ability to share resources (memory, I/O devices, interrupt systems, etc) with interconnection using a bus/crossbar.

Heterogeneous multi-core systems

Heterogeneous (or asymmetric) cores promise further performance and efficiency gains, especially in processing multimedia, recognition and networking applications. For example, a big.LITTLE core includes a high-performance core (called 'big') and a low-power core (called 'LITTLE'). There is also a trend towards improving energy-efficiency by focusing on performance-per-watt with advanced fine-grained power management, dynamic voltage and frequency scaling (i.e., in laptops and portable media players).

Software impact

Parallel programming techniques can benefit from multiple cores directly. Some existing parallel programming models such as CilkPlus, OpenMP, OpenHMP, FastFlow, Skandium, MPI, and Erlang can be used on multi-core platforms. Intel introduced a new abstraction for C++ parallelism called TBB. Other research efforts include the Codeplay Sieve System, Cray's Chapel, Sun's Fortress, and IBM's X10.

Limitations

Having said that, multi-core systems do have limitations in terms of speed gained and effective use of available multiple cores (refer to Figure 2). As software developers, we will be expected to enable a single application to exploit an enormous number of cores that are increasingly diverse (being specialised for different tasks) and at multiple locations (from local to very remote; either on-die, in-box, on-premises or in-cloud). The increasing heterogeneity will continue to spur a deep and fast evolution of mainstream software development. We can attempt to predict some of the changes that could occur.

Amdahl's Law

The original idea presented by Gene Amdahl is a general observation about the performance improvement limits of any